

03 Multimodal Large Model: Autonomous Cruise

1. Brief Game Instructions

Once the program starts running and WonderEcho Pro announces "I am ready."

Say the wake-up word (the wake-up word depends on the firmware flashed, and by default, the wake-up word for our factory firmware is "Hello, HiWonder") to activate WonderEcho Pro. It will respond with "I am here."

You can control MentorPi's line-following function using voice commands, such as **"Go along the black line."** MentorPi converts the spoken command to text via the large model's ASR API, processes it through the large model, After processing, the large model's TTS system vocalizes the generated response, and the corresponding line-following program is executed.

2. Preparation

2.1 Verifying the WonderEcho Pro Firmware

By default, the wake-up word for the WonderEcho Pro factory firmware is "Hello, Hiwonder." If you wish to change the wake-up word to "MentorPi," please refer to the tutorial titled "03 Firmware Flashing Method" for step-by-step instructions.

If you have previously flashed the device with different firmware—such as the version with the wake word “小幻小幻” or “mentorpi”—you must use the corresponding wake word to activate the device.

For example, if the “小幻小幻” firmware was installed, the correct wake word

would be “小幻小幻”; if the “mentorpi” firmware was installed, the wake word would be “mentorpi.”

In the following tutorial, we will use the default factory wake word “Hello, Hiwonder” for demonstration purposes.

2.2 Obtaining and Configuring the Large Model API Key

By default, the program does not include the configuration of the Large AI Model-related API keys. Before activating features related to the Large AI Model, please refer to the section "02 Multimodal Large Model Applications > 01 Obtaining and Configuring Large Model API Keys" to configure the necessary keys. This step is mandatory and cannot be skipped, as it is crucial for the proper functioning and experience of the large model features.

2.3 Network Configuration

Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet. For detailed instructions on network configuration, please refer to the course “20 Network Configuration Courses\ 01 Network Configuration Instructions”

3. Starting and Stopping the Game

Note:

- ① The input commands must strictly observe case sensitivity and spacing.
- ② The robot must be connected to the internet, either in STA (local network)

mode or AP (direct connection) mode via Ethernet.

- 1) Start the robot and follow the instructions in “8. Remote Tool Installation

successfully activated. At this point, the system will begin recording the user's command.

- 7) You can freely express your commands to control MentorPi's movements—for example, **"Go along the black line."**

```
[line_following-10] [INFO] [1747363327.050191000] [line_following]: Wake up detected, exiting large model tracking.
[vocal_detect-11] [INFO] [1747363328.546106521] [vocal_detect]: asr...
[vocal_detect-11] [INFO] [1747363329.238998679] [vocal_detect]: Recording.....
[vocal_detect-11] [INFO] [1747363331.438736688] [vocal_detect]: Done recording
[vocal_detect-11] [INFO] [1747363332.428241187] [vocal_detect]: asr time: 0.99
[vocal_detect-11] [INFO] [1747363333.112118255] [vocal_detect]: publish asr result:Go along the black line.
```

- 8) When the command line shows the output below, it indicates that the cloud-based large speech model's speech recognition service has successfully processed the user's command audio. The recognition result will be displayed under "publish_asr_result."

```
[vocal_detect-11] [INFO] [1747363332.428241187] [vocal_detect]: asr time: 0.99
[vocal_detect-11] [INFO] [1747363333.112118255] [vocal_detect]: publish asr result:Go along the black line.
```

- 9) When the command line shows the output below, it indicates that the cloud-based large language model has successfully processed the user's command, thought through the instruction, and provided a verbal response ("response"), as well as designed an action that aligns with the user's command meaning.

At this point, the robot sends client requests to the "Autonomous Cruise" node to set the color and start the game.

Note: The response is automatically generated by the large model, ensuring the accuracy of the meaning, though the wording and structure of the reply may vary.

```
[llm_visual_patrol-14] [INFO] [1747363334.629849253] [function_call]: msg:{"action":["line_following('black')"],"response":"Time to blaze a trail along the black line!"}
```

- 10) When the following output appears in the terminal, it indicates that the Raspberry Pi has successfully called the cloud-based voice synthesis service. WonderEcho Pro will then play the audio generated from the "response" in step 9.

```
[llm_visual_patrol-14] [INFO] [1747363334.629849253] [function_call]: msg:{"action": "line_following('black')", "response": "Time to blaze a trail along the black line!"}
```

Note: Once the Autonomous Cruise mode is activated, the program will continue running this function. To stop it, press Ctrl+C in the terminal.

11) When the command line shows the output below, it indicates that one round of interaction has been completed. You can refer to Step 5) to reactivate the voice device using the wake-up word and begin a new round of interaction.

```
[vocal_detect-11] [INFO] [1742374059.361436012] [vocal_detect]: enable_wakeup
```

12) To exit this feature, simply press “Ctrl+C” in the terminal. If it doesn’t close on the first attempt, you may need to press it multiple times.

4. How It Works

Once the game begins, you can freely give commands like “Go along the black line” to control the MentorPi’s autonomous line-following. In this mode, the MentorPi will follow the black line. If it fails to detect the black line for more than 5 seconds or if the system is reawakened, the autonomous cruising task will end, and the system will return to the wake-up state.

For the rules governing text responses and action design by the large model, please refer to the prompt settings within the program. Detailed instructions can be found in the PROMPT section of

/home/ubuntu/ros2_ws/src/large_models/large_models/llm_visual_patrol.py

The actions available in this mode are predefined within the program library and are limited in number. MentorPi’s movements are controlled by calling functions that correspond to action strings designed by the large model. If a command involves an action that is not predefined, the program will be unable to execute it; however,

the large model will still recognize the command and provide a response.

5. Brief Program Analysis

The launch file for this program is located at:

The launch file is saved in:

**/home/ubuntu/ros2_ws/src/large_models/launch/llm_visual_patrol.launch
h.py**

```
10 def launch_setup(context):
11     mode = LaunchConfiguration('mode', default=1)
12     mode_arg = DeclareLaunchArgument('mode', default_value=mode)
13
14     app_package_path = get_package_share_directory('app')
15     large_models_package_path = get_package_share_directory('large_models')
16
17     line_following_node_launch = IncludeLaunchDescription(
18         PythonLaunchDescriptionSource(
19             os.path.join(app_package_path, 'launch/line_following_node.launch.py')
20         )),
21         launch_arguments={
22             'debug': 'true',
23         }.items(),
24     )
25
26     large_models_launch = IncludeLaunchDescription(
27         PythonLaunchDescriptionSource(
28             os.path.join(large_models_package_path, 'launch/start.launch.py')),
29         launch_arguments={'mode': mode}.items(),
30     )
31
32     llm_visual_patrol_node = Node(
33         package='large_models',
34         executable='llm_visual_patrol',
35         output='screen',
36     )
```

◆ Line Following Control

The line_following_node launches the line-following module.

◆ Large Model Control Node

The start command launches the large model control node, enabling functions such as speech detection, semantic understanding, task execution, and response playback.

5.1 Main Control Program Analysis

The source code is located at:

`/home/ubuntu/ros2_ws/src/large_models/large_models/llm_visual_patrol.py`

```
6 import re
7 import time
8 import math
9 import rclpy
10 import ast
11 import threading
12 from speech import speech
13 from rclpy.node import Node
14 from large_models.config import *
15 from geometry_msgs.msg import Twist
16
17 from std_msgs.msg import String, Bool
18 from std_srvs.srv import Trigger, SetBool, Empty
19 from rclpy.executors import MultiThreadedExecutor
20 from rclpy.callback_groups import ReentrantCallbackGroup
21
22 from large_models_msgs.srv import SetModel, SetString
23 from ros_robot_controller_msgs.msg import RGBState, RGBStates, SetPWMServoState, PWMServoState
24
```

◆ Library Files Import

The program imports the following modules and libraries to enable functionalities such as ROS2 node creation, message publishing, service calls, threading, and audio playback:

◆ Creates and initializes the ROS2 node

It initializes the ROS2 environment by calling `rclpy.init()` and creates a node instance by inheriting from the `Node` class.

```
101 class LLMControlMove(Node):
102     def __init__(self, name):
103         rclpy.init()
104         super().__init__(name)
```

◆ Initializing Variables

Define the following variables:

`action`: list of actions

`llm_result`: string containing the response from the large language model

running: flag indicating the running status

result: parsed result data

```
106     self.action = []
107     self.llm_result = ''
108     self.running = True
109     self.action_finish = False
110     self.play_audio_finish = False
```

◆ Publisher and Subscriber Initialization

① tts_text_pub: This publisher publishes text messages to the tts_node/tts_text topic. Sending text to the speech synthesis module: This allows the robot to provide voice feedback to the user.

② mecanum_pub: This publisher sends Twist messages to the /controller/cmd_vel topic. The Twist message includes the robot's linear velocities (x, y) and angular velocity (z) to control its movement.

③ llm_result_callback: Subscribes to the agent_process/result topic, which contains commands generated by the Large Language Model (LLM).

④ play_audio_finish_callback: Subscribes to the tts_node/play_finish topic, which notifies the program when voice playback is complete.

```
69     self.mecanum_pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)
70     self.sonar_rgb_pub = self.create_publisher(RGBStates, 'sonar_controller/s
et_rgb', 10)
71     self.rgb_pub = self.create_publisher(RGBStates, 'ros_robot_controller/set
_rgb', 10)
72
73     timer_cb_group = ReentrantCallbackGroup()
74     self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
75
76     self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wake
up')
77     self.create_subscription(String, 'agent_process/result', self.llm_result_
callback, 1)
78     self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_fi
nish_callback, 1, callback_group=timer_cb_group)
79
80     self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wake
up')
81     self.awake_client.wait_for_service()
82     self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callbac
k, 1)
83
```

◆ Determine whether the input is in Chinese

```
25 if os.environ["ASR_LANGUAGE"] == 'Chinese':
26     PROMPT = ''
```

◆ Send Prompt

The PROMPT variable defines the prompt sent to the LLM. The LLM parses user input and generates JSON-formatted output containing actions and responses.

```
53 # Role
54 You are an intelligent vision Mecanum wheel car that needs to generate corresponding JSON instructions based on input commands.
55
56 ## Requirements and Constraints
57 1. Based on the input action description, find the corresponding command in the action function library and output the matching instruction.
58 2. Craft a concise (10-30 words), witty and varied feedback message for each action sequence to make the interaction more engaging.
59 3. Output only the JSON result directly - no analysis or extra content.
60 4. Format: {'action': ['xx', 'xx'], 'response': 'xx'}
61
62 ## Structure Requirements:
63 - The "action" key contains an array of function name strings in execution order. Output [] when no matching action is found.
64 - The "response" key contains a carefully crafted short reply that perfectly matches the word count and style requirements.
65
66
67 ### Task Examples:
68 Input: Follow the black line forward...
69 Output: {"action": ["line_following('black')"], "response": "Tracking black line, smooth sailing ahead!"}
70
71 Input: Follow the red line at your feet, march forward bravely.
72 Output: {"action": ["line_following('red')"], "response": "Red line tracking activated, walking the red carpet like a star!"}
73 ''
```

◆ init_process Method

① Sets the LLM prompt (PROMPT) and stops the mecanum wheeled robot. Play the Startup Audio. Launches a new thread to run the process method. Creates a ROS 2 service ~/init_finish to signal node initialization completion.

② Iterates through the action list, invoking corresponding ROS 2 services based on each action type.

③ For the "line_following" action, extracts color information and calls the related service to start line tracking.

④ After completing all actions, resets the interrupt flag, clears the llm_result variable, and re-enables voice wake-up.

```
107 def init_process(self):
108     self.timer.cancel()
109     msg = SetString.Request()
110     msg.data = PROMPT
111     self.set_prompt_client.call_async(msg)
112     #self.send_request(self.set_prompt_client, msg)
113     self.mecanum_pub.publish(Twist())
114     speech.play_audio(start_audio_path)
115
116     threading.Thread(target=self.process, daemon=True).start()
117
118     self.create_service(Trigger, '~/init_finish', self.get_node_state)
119     self.get_logger().info('\033[1;32m%s\033[0m' % 'start')
```

◆ Main Function

Main Function Execution

```
214 def main():
215     node = FunctionCall('function_call')
216     try:
217         rclpy.spin(node)
218     except KeyboardInterrupt:
219         print('shutdown')
220     finally:
221         rclpy.shutdown()
222
223 if __name__ == "__main__":
224     main()
```

5.2 Large Model Line-Following Logic Analysis

◆ Large Model Interaction Logic Analysis

The configuration file of this program is saved in:

/home/ubuntu/ros2_ws/src/app/app/line_following.py

Used to receive target color information from the large model. The large model sends the target color name to this node by calling the `~/set_large_model_target_color` service.

```
291 def set_target_color_srv_callback(self, request, response):
292     self.get_logger().info('\033[1;32m%s\033[0m' % "set_target_color")
293     with self.lock:
294         x, y = request.data.x, request.data.y
295         self.follower = None
296         if x == -1 and y == -1:
297             self.color_picker = None
298         else:
299             self.color_picker = ColorPicker(request.data, 5)
300             self.mecanum_pub.publish(Twist())
301     response.success = True
302     response.message = "set_target_color"
303     return response
304
305 def get_target_color_srv_callback(self, request, response):
306     self.get_logger().info('\033[1;32m%s\033[0m' % "get_target_color")
307     response.success = False
308     response.message = "get_target_color"
309     with self.lock:
310         if self.follower is not None:
311             response.success = True
312             rgb = self.follower.target_rgb
313             response.message = "{} {}, {}".format(int(rgb[0]), int(rgb[1]), in
t(rgb[2]))
314     return response
315
```

◆ Image_callback Callback Function

① Target Loss Detection When deflection_angle is None, it means the target line is not detected. The follower.lost_target_count counter is incremented. If large_model_tracking is True and the lost count exceeds a predefined threshold, the target is considered lost, triggering a timer to stop the robot after a delay.

② Target Reacquisition Detection When deflection_angle is not None, the target line has been reacquired. The follower.lost_target_count counter is reset, and any active target loss timer is canceled.

③ Motion Control Robot movement is adjusted based on the PID controller output only when deflection_angle is valid, is_running is True, and no obstacles are detected.

```

25 if os.environ["ASR_LANGUAGE"] == 'Chinese':
26     PROMPT = '''
27 # 角色
28 你是一款智能视觉麦轮小车，需要根据输入的内容，生成对应的json指令。
29
30 ##要求与限制
31 1.根据输入的动作内容，在动作函数库中找到对应的指令，并输出对应的指令。
32 2.为动作序列编织一句精炼（10至30字）、风趣且变化无穷的反馈信息，让交流过程妙趣横生。
33 3.直接输出json结果，不要分析，不要输出多余内容。
34 4.格式：{'action':['xx', "xx"], 'response':'xx'}
35
36 ##结构要求##：
37 - "action"键下承载一个按执行顺序排列的函数名称字符串数组，当找不到对应动作函数时action输出[]。
38 - "response"键则配以精心构思的简短回复，完美贴合上述字数与风格要求。
39
40
41 ### 任务示例：
42 输入:沿着黑色的线前行。
43 输出:{"action":["line_following('black')"],"response":"沿着黑线前行，一路无阻!"}
44
45 输入:寻黑色的线
46 输出:{"action":["line_following('black')"],"response":"沿着黑线前行，一路无阻!"}
47
48 输入:沿着脚下的红线，勇往直前。
49 输出:{"action":["line_following('red')"],"response":"红线追踪模式启动，跟着红毯走星光大道"}
50 '''
    
```

◆ Automatic Stop Mechanism

① The start_stop_timer method creates a timer that calls the stop_after_lose method after 5 seconds.

② The stop_after_lose method stops the robot's movement and sets the is_running flag to False, thereby stopping the line-following process.

```

454     def start_stop_timer(self):
455         """Starts the timer to stop the robot after 5 seconds if the target is lost."""
456         if self.target_lost_timer is None: # prevent creating multiple timers
457             self.target_lost_timer = threading.Timer(5.0, self.stop_after_lose)
458             self.target_lost_timer.start()
459             self.target_lost = True
460             # self.mecanum_pub.publish(Twist()) #stop moving
461
462     def stop_after_lose(self):
463         """Stops the robot after the target is lost for 5 seconds."""
464         with self.lock:
465             if self.large_model_tracking:
466                 self.get_logger().warn("目标丢失超过5秒，停止移动 (Target lost for more th
an 5 seconds, stopping movement)")
467                 self.mecanum_pub.publish(Twist()) # 发布停止消息 (publish stop message)
468                 self.is_running = False # 停止巡线 (stop line following)
469                 self.target_lost_timer = None
470                 self.target_lost = False

```

◆ Wake-up Stop

Upon detecting a voice wake-up signal, the robot will stop only if large_model_tracking is set to True. This ensures that the stop command triggered by the voice wake-up signal is executed solely when line-following is active under the large model's control.

```

472     def wakeup_callback(self, msg):
473         """Callback function for the /vocal_detect/wakeup topic."""
474         if msg.data:
475             self.get_logger().info("Wake-up detected, exiting large model tracking.")
476             with self.lock:
477                 if self.large_model_tracking:
478                     self.large_model_tracking = False
479                     self.is_running = False
480                     self.mecanum_pub.publish(Twist()) # Stop the robot
481                     if self.target_lost_timer is not None:
482                         self.target_lost_timer.cancel()
483                         self.target_lost_timer = None
484                     self.follower = None
485             self.get_logger().info("Large model tracking stopped.")

```